

Julia for Finance — Study Guide

Sep 27, 2026 · @Jim Northey

Why Julia for finance, and a learning path

Julia gives you Python-like syntax with C-like speed, so pricing loops, Monte Carlo simulations and risk calculations run fast without vectorization tricks or rewriting in C++. For someone already fluent in Python, the fastest route is: learn the syntax differences, get a working environment, then rebuild familiar finance tasks in Julia.

Week	Focus	Outcome
1	Install Julia, REPL, package manager, Jupyter kernel	Working notebook with Julia kernel
2	Core syntax vs Python, 1-based arrays, broadcasting (.), types	Can translate small Python scripts
3	Multiple dispatch, structs, performance tips (type stability)	Write fast, idiomatic functions
4	DataFrames.jl, CSV.jl, dates, plotting	Load and chart price data
5	Time value of money, bonds, yield curves (FinanceModels.jl)	Bond pricer and curve fit
6	Returns, portfolio optimization (JuMP.jl), Monte Carlo option pricing	Mini portfolio / options notebook

The key mental shifts from Python: arrays start at 1, loops are fast (no need to vectorize everything), functions are organized by **multiple dispatch** rather than classes, and the first call to a function is slower because Julia compiles it.

Setting up a Julia environment

Install Julia with **juliaup**, the official version manager, then create one project environment per piece of work. Official instructions: [Installing Julia](#) and [juliaup on GitHub](#).

1. Install juliaup and Julia

macOS / Linux

```
curl -fsSL https://install.julialang.org | sh
```

Windows (PowerShell) – or install "Julia" from the Microsoft Store

```
winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

Useful juliaup commands:

```
juliaup status      # list installed versions
juliaup update      # update to newest release
juliaup add lts      # add the long-term-support release
juliaup default release # choose which version `julia` starts
```

2. Learn the four REPL modes

Start julia in a terminal. The prompt changes by the key you press:

Key	Mode	Use
(none)	julia>	Run code
]	pkg>	Add, remove, update packages
?	help?>	Documentation for any function
;	shell>	Run shell commands

Press Backspace to return to julia>.

3. Use a project environment (Julia's equivalent of a venv)

Each project keeps its own `Project.toml` (direct dependencies) and `Manifest.toml` (exact locked versions), similar to `pyproject.toml` + a lock file.

```
# in the pkg> prompt
pkg> activate ~/julia-finance      # create/activate an environment in that folder
pkg> add DataFrames CSV Dates Plots FinanceModels
pkg> status                        # list what's installed
pkg> instantiate                  # recreate env from Project/Manifest on another machine
pkg> up                          # update packages
```

Or from code: `using Pkg; Pkg.activate("."); Pkg.add("DataFrames")`. Starting Julia with `julia --project=.` activates the environment in the current folder automatically.

4. Editor and quality-of-life setup

- **VS Code + Julia extension** — the main IDE: inline results, plot pane, debugger, workspace viewer. [Julia in VS Code](#)
- **Revise.jl** — reloads code you edit without restarting Julia. Add it to your global environment and put using `Revise` in `~/.julia/config/startup.jl`.
- **Threads** — start with `julia --threads=auto` to use all cores for parallel Monte Carlo work.
- **Startup time** — the first using `Plots` or first function call compiles code; later calls are fast. Recent Julia versions cache much of this compilation.

Using Julia in Jupyter notebooks

Install the **IJulia** package and a "Julia" kernel appears in Jupyter alongside Python. (The "Ju" in Jupyter is Julia.) Docs: [IJulia.jl on GitHub](#) and [IJulia manual](#).

Option A — use your existing Jupyter / JupyterLab

```
julia> ]
pkg> add IJulia      # install in the global (default) environment
```

Then launch Jupyter as usual (`jupyter lab` or `jupyter notebook`) and choose **Julia 1.x** as the kernel.

Option B — let IJulia install a private Jupyter

```
using IJulia
notebook()      # classic notebook; on first run offers to install Jupyter via Conda
jupyterlab()    # JupyterLab instead
notebook(dir=~/.julia-finance) # start in a specific folder
```

This Miniconda-based Jupyter is kept private to Julia and does not touch your system Python.

Tips

- **Use a project environment inside a notebook** — first cell:

```
using Pkg
Pkg.activate(@__DIR__) # or Pkg.activate("~/julia-finance")
Pkg.instantiate()
```

- **Multithreaded kernel** — create a second kernel that starts with more threads:

```
using IJulia
IJulia.installkernel("Julia 8 threads", env=Dict("JULIA_NUM_THREADS"=>"8"))
```

- **After upgrading Julia** — run `pkg> build IJulia` so the kernel points to the new version.
- **VS Code** opens `.ipynb` files directly with the Julia kernel — no browser needed.
- **Calling Python from Julia** in the same notebook: `PythonCall.jl` lets you `pyimport("pandas")` — handy while migrating.

Alternative: Pluto.jl reactive notebooks

[Pluto.jl](#) is a Julia-native notebook: change a cell and every dependent cell re-runs, like a spreadsheet. Notebooks are plain `.jl` files and each one records its own packages. It's excellent for teaching and "what-if" finance models (sliders for rates, volatility, horizon).

```
pkg> add Pluto
julia> using Pluto; Pluto.run()
```

Python vs Julia syntax comparison

Most Python code translates line-for-line; the traps are 1-based indexing, end blocks instead of indentation, and the dot `.` for element-wise operations. The official [Noteworthy differences from Python](#) page is the canonical reference, and the [QuantEcon MATLAB-Python-Julia cheatsheet](#) is a good printout.

Basics

Task	Python	Julia
Comment	# note	# note (block: <code>#= ... =#</code>)
Print	<code>print(f"NPV = {npv:.2f}")</code>	<code>println("NPV = \$(round(npv, digits=2))")</code> or <code>@printf("NPV = %.2f\n", npv)</code>
Exponent	<code>1.05 ** 10</code>	<code>1.05 ^ 10</code>
Integer division	<code>7 // 2</code>	<code>div(7, 2)</code> or <code>7 ÷ 2</code>
Rational number	<code>Fraction(1, 3)</code>	<code>1//3</code>
String concat	<code>"a" + "b"</code>	<code>"a" * "b"</code>
None / null	<code>None</code>	nothing (missing data: <code>missing</code>)
Booleans	<code>True</code> , <code>False</code> , <code>and</code> , <code>or</code> , <code>not</code>	<code>true</code> , <code>false</code> , <code>&&</code> , <code> </code> , <code>!</code> (see note)
Unicode names	not idiomatic	<code>σ = 0.2</code> (type <code>\sigma + Tab</code>)
Import	<code>import numpy as np</code>	<code>using Statistics</code> or <code>import Statistics as St</code>

Note: Julia's logical OR is two vertical bars (a || b); shown as || in the table above only because a bar would break the table.

Control flow and functions

Task	Python	Julia
If	if x > 0: ... elif ... else:	if x > 0 ... elseif ... else ... end
Ternary	a if cond else b	cond ? a : b
For loop	for t in range(1, 11):	for t in 1:10 ... end
While	while bal < target:	while bal < target ... end
Function	def pv(cf, r, t): return cf / (1+r)**t	pv(cf, r, t) = cf / (1+r)^t
Multi-line function	def f(x): + indented body	function f(x) ... end (last expression is returned)
Default / keyword args	def f(x, r=0.05, *, freq=1)	f(x, r=0.05; freq=1) (keywords after ;)
Lambda	lambda x: x**2	x -> x^2
Comprehension	[x**2 for x in xs if x > 0]	[x^2 for x in xs if x > 0]
Generator sum	sum(cf/(1+r)**t for t, cf in enumerate(cfs, 1))	sum(cf/(1+r)^t for (t, cf) in enumerate(cfs))
Exceptions	try: ... except ValueError as e:	try ... catch e ... end

Arrays (NumPy vs built-in Julia arrays)

Task	Python / NumPy	Julia
Create vector	np.array([1.0, 2.0, 3.0])	[1.0, 2.0, 3.0]
Matrix	np.array([[1, 2], [3, 4]])	[1 2; 3 4]
First / last element	x[0], x[-1]	x[1], x[end]
Slice	x[1:4] (items 2-4)	x[2:4] (inclusive both ends)
Range	np.arange(0, 1, 0.1)	0:0.1:0.9 or range(0, 1, length=11)
Zeros / ones	np.zeros((3, 3))	zeros(3, 3), ones(3)
Element-wise ops	x * y, np.exp(x)	x .* y, exp.(x) (the dot broadcasts)
Whole expression	—	@. p * exp(-r * t) (dots every operation)
Matrix multiply	A @ B	A * B
Transpose	A.T	A' (adjoint) or transpose(A)
Solve linear system	np.linalg.solve(A, b)	A \ b
Append	lst.append(x)	push!(v, x) (! = mutates its argument)
Length / shape	len(x), x.shape	length(x), size(x)
Stats	np.mean(x), np.std(x)	using Statistics; mean(x), std(x)
Cumulative	np.cumsum(x), np.cumprod(1 + r)	cumsum(x), cumprod(1 .+ r)
Random normals	np.random.default_rng(42).standard_normal(n)	randn(MersenneTwister(42), n)

Task	Python / NumPy	Julia
Copy vs view	<code>y = x.copy()</code> , slices are views	<code>y = copy(x)</code> ; slices copy — use <code>@view x[2:4]</code> for a view

Data structures and types

Task	Python	Julia
Dict	<code>{"AAPL": 0.4, "MSFT": 0.6}</code>	<code>Dict{"AAPL" => 0.4, "MSFT" => 0.6}</code>
Tuple / named tuple	<code>(1, 2)</code> , <code>namedtuple</code>	<code>(1, 2)</code> , <code>(ticker="AAPL", qty=100)</code>
Class / record	<code>@dataclass class Bond:</code> <code>coupon: float</code>	<code>struct Bond;</code> <code>coupon::Float64; end</code> (mutable struct to allow changes)
Method on object	<code>bond.price(r)</code>	<code>price(bond, r)</code> — functions live outside the type
Polymorphism	subclass + override method	multiple dispatch: <code>price(b::Bond, r)</code> , <code>price(o::Option, r)</code>
Type hints	optional, not enforced	optional, enforced and used for speed/dispatch
Symbols	—	<code>:close</code> (lightweight names, used for DataFrame columns)

DataFrames (pandas vs DataFrames.jl)

Task	pandas	DataFrames.jl
Read CSV	<code>pd.read_csv("px.csv")</code>	<code>CSV.read("px.csv", DataFrame)</code>
Column	<code>df["close"]</code>	<code>df.close</code> or <code>df[!, :close]</code>
Filter rows	<code>df[df.close > 100]</code>	<code>filter(:close => >(100), df)</code> or <code>df[df.close .> 100, :]</code>
New column	<code>df["ret"] =</code> <code>df.close.pct_change()</code>	<code>df.ret = [missing;</code> <code>diff(df.close) ./</code> <code>df.close[1:end-1]]</code>
Group + aggregate	<code>df.groupby("ticker")["ret"].mean()</code>	<code>combine(groupby(df, :ticker), :ret => mean)</code>
Sort	<code>df.sort_values("date")</code>	<code>sort(df, :date)</code>
Head	<code>df.head()</code>	<code>first(df, 5)</code>
Pipe-style	method chaining	<code>@chain df begin ... end</code> (DataFramesMeta.jl)

For a full side-by-side, see [DataFrames.jl: comparison with pandas](#).

The same NPV function in both languages

Python

def npv(rate, cashflows):

```

    return sum(cf / (1 + rate) ** t for t, cf in enumerate(cashflows))

npv(0.08, [-1000, 300, 400, 500])
# Julia
npv(rate, cashflows) = sum(cf / (1 + rate)^(t-1) for (t, cf) in enumerate(cashflows))

npv(0.08, [-1000, 300, 400, 500])

```

Note the $t-1$: enumerate starts at 1 in Julia, so the first cash flow is at time 0 only after adjusting.

Key Julia packages for finance

The finance stack is smaller than Python's but strong in fixed income, actuarial work, optimization and simulation. Browse more at [Julia Packages: Finance](#) and the [JuliaQuant](#) and [JuliaActuary](#) organizations.

Area	Package	Python analogue	What it's for
Tabular data	DataFrames.jl , CSV.jl	pandas	Load, clean, group, join data
Dates / calendars	Dates (built in), BusinessDays.jl	datetime, pandas offsets	Day counts, holiday calendars
Time series	TimeSeries.jl	pandas time series	TimeArray, lags, moving windows
Market data	YFinance.jl , MarketData.jl	yfinance, pandas-datareader	Prices, fundamentals, options chains, FRED data
Fixed income / valuation	FinanceModels.jl	QuantLib-Python	Yield curves, bond and option pricing, curve fitting
Financial math	ActuaryUtilities.jl	numpy-financial	NPV, IRR, duration, convexity, breakeven
Statistics	Statistics, StatsBase.jl , Distributions.jl	scipy.stats	Moments, quantiles, VaR, sampling
Regression	GLM.jl	statsmodels	CAPM / factor regressions
Optimization	JuMP.jl + HiGHS / Ipopt, Optim.jl	cvxpy, scipy.optimize	Mean-variance portfolios, calibration
Autodiff	ForwardDiff.jl	JAX	Exact Greeks and sensitivities
Simulation / SDEs	DifferentialEquations.jl	—	Stochastic processes, interest-rate models
Plotting	Plots.jl , Makie.jl	matplotlib, plotly	Charts, interactive plots
Python bridge	PythonCall.jl	—	Use pandas, QuantLib etc. from Julia
Economics	QuantEcon.jl	quantecon	Markov chains, dynamic programming

A starter environment for this guide:

```

pkg> activate ~/julia-finance
pkg> add IJulia DataFrames CSV TimeSeries YFinance FinanceModels ActuaryUtilities StatsBase Dist

```

Online references and courses

Start with the official manual's [Python-differences](#) page and QuantEcon's [Julia lectures](#); for finance specifically, *Modern Financial Modeling* and Cornell's CHEME 5660 materials are the best free resources. All below are free.

Resource	Type	Why use it
Julia Manual	Official docs	The reference; read "Getting Started" through "Methods"
Noteworthy differences from Python	Official docs	Must-read for Python users
Performance Tips	Official docs	Why your code is slow (type stability, globals)
Modern Julia Workflows	Guide	Environments, VS Code, testing, packaging done right
QuantEcon MATLAB-Python-Julia cheatsheet	Cheatsheet	Side-by-side syntax printout
Quantitative Economics with Julia	Lecture series	Setup, Julia essentials, then economic and asset-pricing models
Modern Financial Modeling (Loudenback & Lee)	Open-access book	Financial modeling in Julia for actuaries and finance professionals
CHEME 5660: Finance and Markets for Engineers and Scientists (Cornell, J. Varner)	Course book + notebooks	Returns, portfolios, options, hedging in Julia; Fall 2026 course repo
JuliaActuary tutorials	Tutorials	FinanceModels.jl and ActuaryUtilities.jl examples
MIT 18.S191 Computational Thinking	University course	Julia + Pluto via real modeling problems; GitHub
Julia Data Science	Free online book	DataFrames and Makie fundamentals
DataFrames.jl vs pandas	Official docs	Translating pandas code
JuliaAcademy	Video courses	Free intro courses from Julia developers
Exercism Julia track	Practice	Small exercises with mentor feedback
Julia Discourse — Finance & Economics	Forum	Ask questions; very responsive community

YouTube videos and channels

For learning the language, [doggo dot jl's](#) series and MIT 18.S191 are the most watchable; for finance applications, pick from the [JuliaCon](#) talks below.

Channels and playlists for learning Julia

Video / channel	Why watch
The Julia Programming Language (official channel)	All JuliaCon talks, workshops and tutorials
doggo dot jl (formerly "Julia for Talented Amateurs")	Clear beginner series: install, VS Code, Pluto, DataFrames, plotting; code on GitHub

Video / channel	Why watch
MIT 18.S191 Computational Thinking — Spring 2021 MIT 18.S191 — Fall 2020	Full lecture course in Julia + Pluto Earlier run of the same course, with Grant Sanderson (3Blue1Brown)
JuliaCon 2025 (Pittsburgh) playlist JuliaCon 2024 (Eindhoven) playlist	Latest talks, incl. workshops for beginners Talks and workshops

Finance and economics talks

Talk	Topic
Academia to Finance: How Julia Makes Efficient Research Possible — Hansen, JuliaCon 2025	Using Julia for finance research workflows
Julia for End-to-End Financial Analysis — Mohammed Zahran, JuliaCon 2021	Data → analysis → reporting pipeline in Julia
Quantitative Macroeconomics in Julia — Tom Sargent (Nobel laureate), JuliaCon 2016 keynote	Why QuantEcon adopted Julia
Miletus: A Financial Modelling Suite in Julia — Anantharaman & Byrne, JuliaCon 2017	Contract-based derivative modeling; the ideas carried into FinanceModels.jl (package itself is dated)
Teaching Quantitative Finance to Engineers using Julia — Jeffrey Varner, JuliaCon 2023	How Cornell's CHEME 5660 teaches quant finance in Julia; useful for your own teaching

Tip: search the official channel for “workshop” — each JuliaCon posts 2–3 hour hands-on sessions on DataFrames, Makie, JuMP and performance.

Books

If you buy one book, get *Julia for Data Analysis*; for finance, read the free *Modern Financial Modeling* alongside it. The official [Julia books list](#) has more.

Book	Author(s), publisher, year	Level	Why read it
Modern Financial Modeling	Alec Loudenback & Yun-Tien Lee, 2026 (free HTML/PDF; print available)	Intermediate	Julia-based financial modeling: computational thinking, portfolio optimization, stochastic projections
Julia for Data Analysis	Bogumił Kamiński, Manning, 2022	Beginner-intermediate	Written by the DataFrames.jl lead developer; code on GitHub
Julia as a Second Language	Erik Engheim, Manning, 2023	Beginner	Aimed at programmers coming from Python and other languages

Book	Author(s), publisher, year	Level	Why read it
Think Julia	Ben Lauwens & Allen B. Downey, O'Reilly, 2019	Beginner	Julia version of the classic <i>Think Python</i>
Hands-On Design Patterns and Best Practices with Julia	Tom Kwong, Packt, 2020	Intermediate-advanced	Multiple dispatch, traits, performance patterns
Statistics with Julia	Hayden Klok & Yoni Nazarathy, Springer, 2021	Intermediate	Probability, inference, regression, simulation
Using Julia for Introductory Econometrics	Florian Heiss & Daniel Brunner, 2023 (free PDF; print on Amazon)	Beginner-intermediate	Econometrics examples; pairs with Wooldridge's textbook
Introduction to Quantitative Macroeconomics Using Julia	Petre Caraiani, Academic Press, 2019	Intermediate	Macro/DSGE models in Julia
Finance and Markets for Engineers and Scientists	Jeffrey Varner, Cornell (free online)	Intermediate	Course text for CHEM 5660: returns, portfolios, options in Julia

Practice projects

Rebuild finance tasks you already know in Python; each project below exercises a different part of the language. Do each one as a Jupyter or Pluto notebook in your ~/julia-finance environment.

1. **Time value of money** — write `pv`, `fv`, `npv`, `irr` from scratch, then check against `ActuaryUtilities.jl`. Practises functions, keyword arguments, generators.
2. **Price history and returns** — pull 5 years of prices with `YFinance.jl`, compute daily log returns, rolling volatility and max drawdown in a `DataFrame`, then plot. Practises `DataFrames`, broadcasting, missing values.
3. **Bond pricer with multiple dispatch** — define struct `FixedBond` and struct `ZeroBond`, write one `price(b, curve)` method per type, then add duration and convexity. Compare with `FinanceModels.jl`. Practises structs and dispatch.
4. **Yield curve** — fit a Nelson-Siegel curve to Treasury par yields with `FinanceModels.jl` or `Optim.jl`.
5. **CAPM / factor regression** — regress a stock's excess returns on market excess returns with `GLM.jl`; report alpha, beta, R^2 .
6. **Mean-variance portfolio** — build the efficient frontier with `JuMP.jl` + `Ipopt` (quadratic objective, long-only constraint).
7. **Monte Carlo option pricing** — price a European call, check against Black-Scholes, then compute delta with `ForwardDiff.jl`. Time the Julia loop against your NumPy version.
8. **Pluto what-if model** — a loan amortization or retirement projection with sliders for rate, term and contributions; well suited as a classroom demo.

Starter code for project 7:

```
using Random, Statistics
```

```
function mc_call(S0, K, r, σ, T; n=1_000_000, rng=Random.default_rng())
    z = randn(rng, n)
    ST = @. S0 * exp((r - σ^2/2)*T + σ*sqrt(T)*z)
```

```
    return exp(-r*T) * mean(max.(ST .- K, 0))  
end
```

```
mc_call(100, 100, 0.05, 0.20, 1.0) # ≈ 10.45 (Black–Scholes: 10.4506)
```

Then try rewriting it as a plain for loop with no arrays — in Julia that version is usually as fast or faster, which is the key difference from Python.